Group 27
Quantum Query



Uber Bolt

Project Phase 1

(CMPG321)

Group Members:

35161124 – K. L – (Group Leader)

46892893 – O. Matlhaku

46359990 – N. Rantsieng

37335944 – T. Mlangeni

41731174 – T. Phasha

42979412 – N. Mbuyisa

Submission date: 16/10/2026

TABLE OF CONTENTS

Group Information 1

Phase 1 – Focus Area and Requirements Analysis 2

1. Area of Focus 2

2. Research Questions 3

3. Dataset Description 5

3.1 Main Tables 5

3.2 Additional Tables (Contextual Data) 6

Phase 2 – DB Implementation and Analysis 7

4. DB Design and Setup 7

4.1 Reference Tables: Zones, Riders, Drivers and Vehicles 8

4.2 Trip Tables: Headers, Fare Breakdown and Reviews 10

4.3 Keys & Constarints 12

4.4 Final Table Model 13

4.5 Importing the Data 17

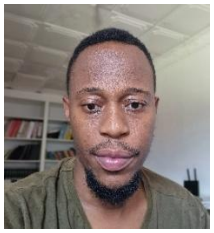
4.6 Power BI Connection and Data Model 18

5. Schema / ERD 19

6. SQL Query Development 20

7. Analysis and Reporting 30

GROUP INFORMATION



Kamogelo Langa (Group Leader)

3561124

35161124@mynwu.ac.za

0793872356

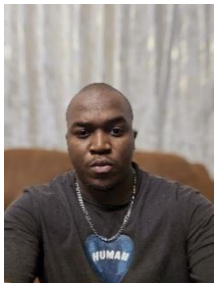


Thabiso Phasha

41731174

41731174@mynwu.ac.za

0790958219



Talani Mlangeni

37335944

37335944@mynwu.ac.za

0626017757



Othusitse Matlhaku

46892893

46892893@mynwu.ac.za

0634521990

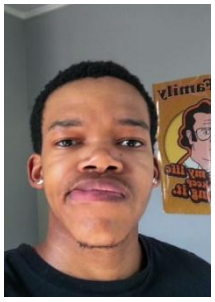


Nokuthula Mbuyisa

42979412

42979412@mynwu.ac.za

0655471231



Neo Rantsieng

46359990

46359990@mynwu.ac.za

0694711889

Phase 1 – Focus Area and Requirements Analysis

1. AREA OF FOCUS

Company Background

The ride-hailing market in South Africa is highly competitive. Big, established companies like Uber and Bolt are fighting for customers against newer apps like InDrive. While riders care a lot about clear prices and safety, the quality of the cars is the main thing that keeps passengers happy, stops drivers from leaving, and builds a strong brand.

Quantum Query was hired by InDrive to look closely at market data to understand their competition (Uber/Bolt). By finding the weaknesses of these top companies in major cities (like Johannesburg, Cape Town, Durban, Gqheberha Potchefstroom, and Pretoria). Quantum Query will give InDrive clear steps to win more customers and improve its fleet of cars.

Strategic Focus: Fleet Quality Control

Maintaining a high standard of vehicle quality and driver reliability directly impacts platform growth. In emerging markets, passenger experience is heavily influenced by ride comfort, vehicle roadworthiness, and brand perception. This consulting engagement focuses on uncovering operational gaps across three core dimensions namely, geographic brand suitability, asset depreciation & passenger sentiment, driver affiliation & fleet longevity.

The analysis is structured around three foundational research pillars designed to inform operational policies:

- **Regional Vehicle Benchmarking:** Identify vehicle makes that consistently underperform city-level baseline ratings, establishing data-backed onboarding restrictions and targeted upgrade incentives by city.
- **Fleet Age Threshold Policy:** Determine empirical turning points where vehicle wear correlates with rating declines, providing the analytical basis for vehicle renewal programs and trade-in partnerships.
- **Market Share & Driver Survival Analysis:** Quantify platform market share across operating cities and compare the operational lifespan (lifetime trips) of dual platform drivers against platform exclusive fleets to optimize onboarding incentives.

The main objective of the analysis is to assist Indrive improve their fleet quality and improve customer satisfaction.

2. RESEARCH QUESTIONS

Question 1: Regional Vehicle Make Performance Benchmarking

- **Business Context:** Passenger expectations, road infrastructure, and trip topographies vary across South African metropolitan areas. A vehicle model that performs well in Johannesburg may underperform in Cape Town or Durban due to distinct terrain, commuter density, or passenger comfort standards. Applying a uniform national fleet standard overlooks localized dissatisfaction driven by specific car brands.
- **Research Question:** How do vehicle manufacturers rank within each specific South African operating city based on average driver ratings, and which specific vehicle makes consistently fall below their respective city-wide baseline rating?
- **Actionable Insight:** InDrive can implement localized vehicle onboarding criteria instead of generic national policies. By identifying vehicle makes that persistently depress city-wide ratings, InDrive can selectively restrict underperforming models in specific cities or offer targeted upgrade subsidies to drivers using those vehicles, directly improving local fleet perception.

Question 2: Asset Age Cutoff and Passenger Satisfaction

- **Business Context:** Physical wear-and-tear, interior degradation, and mechanical reliability naturally worsen as vehicles age. For ride-hailing platforms, older cars often lead to lower passenger comfort, increased noise, and higher maintenance risks, which riders reflect through lower trip ratings. Determining the precise age threshold at which ratings drop allows the platform to act before customer satisfaction degrades.
- **Research Question:** How does vehicle age (categorized into brackets: 0–2, 3–5, 6–8, and 9+ years) affect average driver ratings across the fleet?
- **Actionable Insight:** The platform can establish an empirical cut-off point for fleet onboarding and renewal policies. If vehicles older than 8 years show a sharp, measurable drop in ratings, InDrive can introduce a mandatory maximum vehicle age cap for premium tiers, negotiate trade-in discounts with automotive partners, and offer renewal incentives for drivers approaching the critical age threshold.

Question 3: Platform Affiliation Dynamics and Driver Operational Longevity

- **Business Context:** High driver turnover is a persistent challenge in the South African e-hailing sector due to tight profit margins and platform switching. Drivers frequently "multi-app" between Uber and Bolt to minimize idle time. Understanding whether dual-platform drivers sustain longer operational lifespans than single-platform drivers is critical for designing competitive retention and loyalty programs.
- **Research Question:** What is the relative percentage market share of each platform affiliation (Uber, Bolt, and Dual-platform) within each South African operating city, and how does the average lifetime trip count of Dual-platform drivers compare to the baseline of platform-exclusive drivers?

- **Actionable Insight:** If dual-platform drivers demonstrate greater operational longevity (measured by lifetime trips), InDrive can strategically target this flexible segment with lower commission structures and flexible dispatch policies. This allows InDrive to capture active supply from Uber and Bolt without demanding strict exclusivity, stabilizing fleet availability in high-churn cities.

Question 4: How does vehicle age and ride category influence rider tipping behavior as a proxy for customer satisfaction?

- **Business Context:** While 1-to-5 star ratings can sometimes be arbitrary, a rider's willingness to tip is a definitive, monetary indicator of exceptional passenger satisfaction.
- **Research Question:** What is the average tip amount (as a percentage of the total fare) grouped by vehicle age brackets (0–3, 4–7, 8+ years) and vehicle category types, and does this financial sentiment differ between Uber and Bolt platforms?
- **Actionable Insight:** By proving that newer or specific vehicle categories yield significantly higher tips, InDrive can use this data as a marketing tool to recruit top-tier drivers. It tells drivers: "Upgrade your car and move to InDrive to increase your take-home pay via tips."

Question 5: Does a higher platform commission rate degrade fleet quality over time?

- **Business Context:** High commission platforms often squeeze driver earnings, which may leave drivers with less disposable income to spend on vehicle maintenance and upgrades. This is a critical weakness InDrive (which typically offers lower commissions) can exploit.
- **Research Question:** Is there a direct, correlation between a driver’s platform commission (grouped by Uber vs. Bolt exclusives) and the degradation of their fleet quality (measured by average vehicle age and average driver rating)?
- **Actionable Insight:** If the analysis reveals that platforms extracting higher commissions result in an aging, lower-rated fleet, InDrive can heavily promote its lower-commission model to the public as the reason its cars are cleaner, newer, and safer.

3. DATASET DESCRIPTION

3.1 MAIN TABLES

Vehicles (vehicles.csv)

- **Key Fields for Analysis:**
    - ***make* (Categorical/String)**: The brand of the vehicle (e.g., Toyota, Suzuki, VW). *Essential for benchmarking brand performance across cities*.
    - ***year* (Integer):** The manufacturing year ranging from 2016 to 2025. *Essential for deriving the "vehicle age bracket" metric to analyze physical degradation*.
    - ***category_type* (Categorical/String):** The tier of the vehicle (e.g., Standard, Comfort).
    - ***vehicle_id*** & ***driver_id*** (Primary/Foreign Keys).

Drivers (sa_drivers.csv)

- **Key Fields for Analysis:**
    - ***driver_rating* (Float/Numeric):** The aggregated lifetime rating for the driver, ranging from 4.1 to 5.0. *Essential for matching vehicle quality to passenger satisfaction*.
    - ***platform_affiliation* (Categorical/String)**: Indicates if the driver is exclusive to Uber, Bolt, or Dual-Platform. *Critical for analyzing retention and multi-apping dynamics*.
    - **total_lifetime_trips (Integer)**: Indicates operational longevity, ranging from 181 to 6,795 trips.
    - ***operating_city* (Categorical/String)**: The localized city of operation (e.g., Gqeberha, Durban, Cape Town, Potchefstroom). *Crucial for city-level quality isolation*.

Trip Headers (trip_headers.csv)

- **Key Fields for Analysis:**
    - ***trip_status* (Categorical/String):** Indicates if a trip was 'COMPLETED', 'CANCELLED_BY_RIDER', or 'CANCELLED_BY_DRIVER'.
    - ***cancellation_reason* (Categorical/String):** Contains nulls for completed trips, but holds valuable data (e.g., 'SAFETY_CONCERN', 'DRIVER_TOO_FAR') for cancelled trips.
    - ***driver_id*** & ***rider_id*** (Foreign Keys): Used to link rider demographics and vehicle/driver quality to a specific ride event.

Trip Reviews (trip_reviews.csv)

- **Key Fields for Analysis:**
    - rider_rating_of_driver (Integer): Scale of 1 to 5. Unlike the aggregated driver rating, this represents trip-specific satisfaction.
    - feedback_comment (Categorical/String): Qualitative feedback text (e.g., "Vehicle aircon was off on a hot day.") which directly informs fleet comfort levels.

3.2 Additional Tables (Contextual Data)

- **Trip Fare Breakdown (trip_fare_breakdown.csv):** 50,000 records detailing tip_zar, platform_commission_zar, and distance_km. The tip amount serves as a proxy for exceptional rider satisfaction.
- **Pricing Surge Zones (pricing_surge_zones.csv):** 9 zones mapping zone_id to specific metropolitan areas and capturing regional base fares.
- **SA Riders (sa_riders.csv):** 1,200 records outlining passenger profiles, home_city, and preferred_payment methods.

Phase 2 – DB Implementation and Analysis

4. DB Design and Setup

The relational database is a PostgreSQL schema of seven tables that records South African ride-hailing trips (Uber and Bolt) in rand (ZAR). Three tables describe the people and prices (riders, drivers, surge zones), one describes the vehicle, and three record each trip: who travelled, what it cost, and how it was rated.

Table	**What one row holds**	**Primary key**	**Linked to**
pricing_surge_zones	A pricing zone with its base fare and per-km and per-minute rates	zone_id	trip_headers
sa_riders	A rider: name, city, province, preferred payment	rider_id	trip_headers
sa_drivers	A driver: city, platform, commission, rating, lifetime trips	driver_id	vehicles, trip_headers
vehicles	The one vehicle a driver uses	vehicle_id	sa_drivers (one per driver)
trip_headers	A trip request: rider, driver, zone, category, status	trip_id	sa_riders, sa_drivers, pricing_surge_zones
trip_fare_breakdown	The money side of one trip: distance, surge, tip, commission, payout	trip_id	trip_headers (one to one)
trip_reviews	The ratings and comment for one trip	review_id	trip_headers (one to one)

4.1 Reference Tables: Zones, Riders, Drivers and Vehicles

```
CREATE TABLE IF NOT EXISTS pricing_surge_zones (
    zone_id          VARCHAR(50) PRIMARY KEY,
    zone_name        VARCHAR(100),
    city             VARCHAR(50),
    base_fare_zar    NUMERIC(10, 2),
    per_km_rate_zar  NUMERIC(10, 2),
    per_min_rate_zar NUMERIC(10, 2)
);
```

- zone_id is the primary key and is text, so the source system's zone codes can be kept as they are.
- The three rate columns use NUMERIC(10, 2): exact decimals with two places. NUMERIC is used for all money in this database because floating-point types can produce rounding errors in sums.

```
CREATE TABLE IF NOT EXISTS sa_riders (
    rider_id              VARCHAR(30) PRIMARY KEY,
    first_name            VARCHAR(50),
    last_name             VARCHAR(50),
    mobile_number         VARCHAR(25),
    home_city             VARCHAR(50),
    province              VARCHAR(50),
    preferred_payment     VARCHAR(30),
    account_created_date  DATE
);
```

- rider_id is the primary key.
- mobile_number is text, not a number, so leading zeros and + signs survive.
- home_city and province allow riders to be grouped geographically, and preferred_payment records their usual payment method.

```sql
CREATE TABLE IF NOT EXISTS sa_drivers (
    driver_id               VARCHAR(30) PRIMARY KEY,
    first_name              VARCHAR(50),
    last_name               VARCHAR(50),
    operating_city          VARCHAR(50),
    platform_affiliation    VARCHAR(50),
    platform_commission_pct NUMERIC(5, 4),
    driver_rating           NUMERIC(3, 2),
    total_lifetime_trips    INTEGER,
    onboarding_date         DATE
);
```

- operating_city and platform_affiliation (Uber, Bolt or Dual-Platform (Both)) drive the market share and longevity analysis in Question 3.
- platform_commission_pct is NUMERIC(5, 4) and holds a fraction such as 0.2500 for 25%. That is why Question 5 multiplies it by 100.
- driver_rating is NUMERIC(3, 2), for example 4.55. It is the average rating used in Questions 1, 2 and 5.
- total_lifetime_trips is a whole number and is the longevity measure in Question 3. onboarding_date records when the driver joined.

```sql
CREATE TABLE IF NOT EXISTS vehicles (
    vehicle_id      VARCHAR(50),
    driver_id       VARCHAR(30),
    make            VARCHAR(50),
    model           VARCHAR(50),
    year            INTEGER,
    license_plate   VARCHAR(30),
    category_type   VARCHAR(60)
);
```

- year is the model year (an integer), from which vehicle age is calculated as the current year minus year.
- make, model and category_type describe the car. License plate is stored for identification.
- The primary key and the unique rules are added afterwards

4.2 Trip Tables: Headers, Fare Breakdown and Reviews

```
CREATE TABLE IF NOT EXISTS trip_headers (
    trip_id              VARCHAR(40),
    rider_id             VARCHAR(30),
    driver_id            VARCHAR(30),
    zone_id              VARCHAR(50),
    ride_category        VARCHAR(60),
    request_timestamp    TIMESTAMP,
    trip_status          VARCHAR(40),
    cancellation_reason  VARCHAR(60)
);
```

- One row per trip request. rider_id, driver_id and zone_id are the three links to the master tables.
- ride_category (for example Standard, Comfort, Budget, Large Capacity) is the service level used in Question 4.
- request_timestamp is a TIMESTAMP, so trips can be analysed by hour, day or season.
- trip_status separates COMPLETED trips from cancelled ones, and cancellation_reason is filled only when a trip was cancelled.

```
CREATE TABLE IF NOT EXISTS trip_fare_breakdown (
    trip_id                  VARCHAR(40),
    distance_km              NUMERIC(8, 2),
    duration_minutes         INTEGER,
    surge_multiplier         NUMERIC(4, 2),
    base_fare_zar            NUMERIC(10, 2),
    tolls_zar                NUMERIC(10, 2),
    tip_zar                  NUMERIC(10, 2),
    total_fare_zar           NUMERIC(10, 2),
    platform_commission_zar  NUMERIC(10, 2),
    driver_payout_zar        NUMERIC(10, 2),
    payment_method           VARCHAR(40)
);
```

- One row per trip, sharing the trip_id with the header.
- distance_km, duration_minutes and surge_multiplier explain how the fare was built, and base_fare_zar and tolls_zar are its parts.
- tip_zar and total_fare_zar give the tip percentage used in Question 4.
- platform_commission_zar and driver_payout_zar show how each fare was split between the platform and the driver.
- payment_method records how the rider paid.

```sql
CREATE TABLE IF NOT EXISTS trip_reviews (
    review_id               VARCHAR(30),
    trip_id                 VARCHAR(40),
    rider_rating_of_driver  SMALLINT,
    driver_rating_of_rider  SMALLINT,
    feedback_comment        TEXT
);
```

- One row per review, with ratings in both directions: the rider rates the driver and the driver rates the rider. SMALLINT is enough for a small rating scale.
- feedback_comment is TEXT, so comments can be any length.
- Each trip can have at most one review

4.3 KEYS & CONSTARINTS

Adding Primary Keys

```
ALTER TABLE vehicles
    ADD CONSTRAINT pk_vehicles PRIMARY KEY (vehicle_id);

ALTER TABLE trip_headers
    ADD CONSTRAINT pk_trip_headers PRIMARY KEY (trip_id);

ALTER TABLE trip_fare_breakdown
    ADD CONSTRAINT pk_trip_fare_breakdown PRIMARY KEY (trip_id);

ALTER TABLE trip_reviews
    ADD CONSTRAINT pk_trip_reviews PRIMARY KEY (review_id);
```

Adding Constraints

```
ALTER TABLE vehicles
    ADD CONSTRAINT uq_vehicles_driver_id UNIQUE (driver_id),
    ADD CONSTRAINT uq_vehicles_license_plate UNIQUE (license_plate);

ALTER TABLE trip_reviews
    ADD CONSTRAINT uq_trip_reviews_trip_id UNIQUE (trip_id);
```

Adding Foreign Keys

```
ALTER TABLE vehicles
    ADD CONSTRAINT fk_vehicles_driver
    FOREIGN KEY (driver_id) REFERENCES sa_drivers (driver_id);

ALTER TABLE trip_headers
    ADD CONSTRAINT fk_trip_headers_rider
    FOREIGN KEY (rider_id) REFERENCES sa_riders (rider_id);

ALTER TABLE trip_headers
    ADD CONSTRAINT fk_trip_headers_driver
    FOREIGN KEY (driver_id) REFERENCES sa_drivers (driver_id);

ALTER TABLE trip_headers
    ADD CONSTRAINT fk_trip_headers_zone
    FOREIGN KEY (zone_id) REFERENCES pricing_surge_zones (zone_id);

ALTER TABLE trip_fare_breakdown
    ADD CONSTRAINT fk_fare_breakdown_trip
    FOREIGN KEY (trip_id) REFERENCES trip_headers (trip_id);

ALTER TABLE trip_reviews
    ADD CONSTRAINT fk_trip_reviews_trip
    FOREIGN KEY (trip_id) REFERENCES trip_headers (trip_id);
```

4.4 Final Table Model

'pricing_surge_zones' Table model:

This table stores the pricing zones that the fares are based on.

Primary Key:

- zone_id: This is the primary key and uniquely identifies a pricing zone.

Columns:

- zone_id: Identifies the pricing zone.
- zone_name: Name of the zone.
- city: The city in which the zone is located.
- base_fare_zar: The starting fare for a trip in the zone, in rand.
- per_km_rate_zar: The rate charged for each kilometre travelled, in rand.
- per_min_rate_zar: The rate charged for each minute of the trip, in rand.

Foreign Key Constraints:

- None. This is a parent table: trip_headers refers to it through fk_trip_headers_zone.

'sa_riders' Table model:

This table stores one record for every rider who has an account.

Primary Key:

- rider_id: This is the primary key and uniquely identifies a rider.

Columns:

- rider_id: Identifies the rider.
- first_name and last_name: The rider's name.
- mobile_number: The rider's phone number, stored as text so that leading zeros and + signs are kept.
- home_city: The city where the rider lives.
- province: The province where the rider lives.
- preferred_payment: The payment method the rider usually uses.
- account_created_date: The date the rider's account was created.

Foreign Key Constraints:

- None. This is a parent table: trip_headers refers to it through fk_trip_headers_rider.

'sa_drivers' Table model:

This table stores one record for every driver, including the platform they work on and their performance.

Primary Key:

- driver_id: This is the primary key and uniquely identifies a driver.

Columns:

- driver_id: Identifies the driver.
- first_name and last_name: The driver's name.
- operating_city: The city in which the driver works.
- platform_affiliation: The platform the driver works on: Uber, Bolt or Dual-Platform (Both).
- platform_commission_pct: The commission the platform takes, stored as a fraction (0.2500 means 25%).
- driver_rating: The driver's average rating from riders.
- total_lifetime_trips: The number of trips the driver has completed over their time on the platform.
- onboarding_date: The date the driver joined.

Foreign Key Constraints:

- None. This is a parent table: vehicles refers to it through fk_vehicles_driver and trip_headers refers to it through fk_trip_headers_driver.

'vehicles' Table model:

This table stores the vehicle that each driver uses.

Primary Key:

- vehicle_id: This is the primary key (pk_vehicles) and uniquely identifies a vehicle.

Columns:

- vehicle_id: Identifies the vehicle.
- driver_id: The driver who uses the vehicle.
- make: The manufacturer, for example Toyota or Kia.
- model: The vehicle model.
- year: The model year, used to work out the vehicle's age.
- license_plate: The vehicle's registration plate.
- category_type: The vehicle category the car falls under.

Unique Constraints:

- uq_vehicles_driver_id: Each driver can have only one vehicle.
- uq_vehicles_license_plate: No two vehicles can share a license plate.

Foreign Key Constraints:

- fk_vehicles_driver: Every driver_id must refer to an existing driver in the sa_drivers table.

'trip_headers' Table:

This table stores one record for every trip request, whether or not the trip was completed.

Table Model:

Primary Key:

- trip_id: This is the primary key (pk_trip_headers) and uniquely identifies a trip.

Columns:

- trip_id: Identifies the trip.
- rider_id: The rider who requested the trip.
- driver_id: The driver assigned to the trip.
- zone_id: The pricing zone the trip belongs to.
- ride_category: The service level requested, for example Standard or Comfort.
- request_timestamp: The date and time the trip was requested.
- trip_status: Whether the trip was completed or cancelled.
- cancellation_reason: Why the trip was cancelled. It is filled only for cancelled trips.

Foreign Key Constraints:

- fk_trip_headers_rider: Every rider_id must refer to an existing rider in the sa_riders table.
- fk_trip_headers_driver: Every driver_id must refer to an existing driver in the sa_drivers table.
- fk_trip_headers_zone: Every zone_id must refer to an existing zone in the pricing_surge_zones table.

'trip_fare_breakdown' Table model:

This table stores the money side of each trip: how the fare was built and how it was split.

Primary Key:

- trip_id: This is the primary key (pk_trip_fare_breakdown) and is also a foreign key, so each trip can have only one fare record.

Columns:

- trip_id: Identifies the trip the fare belongs to.
- distance_km: The distance travelled, in kilometres.
- duration_minutes: The length of the trip, in minutes.
- surge_multiplier: The surge pricing factor applied to the fare.
- base_fare_zar: The base part of the fare, in rand.
- tolls_zar: Toll charges added to the fare, in rand.
- tip_zar: The tip the rider gave, in rand.
- total_fare_zar: The total fare, in rand.
- platform_commission_zar: The part of the fare kept by the platform, in rand.
- driver_payout_zar: The part of the fare paid to the driver, in rand.
- payment_method: How the rider paid.

Foreign Key Constraints:

- fk_fare_breakdown_trip: Every trip_id must refer to an existing trip in the trip_headers table.

'trip_reviews' Table model:

This table stores the ratings and comment left after a trip.

Primary Key:

- review_id: This is the primary key (pk_trip_reviews) and uniquely identifies a review.

Columns:

- review_id: Identifies the review.
- trip_id: The trip the review is about.
- rider_rating_of_driver: The rating the rider gave the driver.
- driver_rating_of_rider: The rating the driver gave the rider.
- feedback_comment: The written comment, of any length.

Unique Constraints:

- uq_trip_reviews_trip_id: Each trip can have at most one review.

Foreign Key Constraints:

- fk_trip_reviews_trip: Every trip_id must refer to an existing trip in the trip_headers table.

4.5 IMPORTING THE DATA

The data was supplied as CSV files, one per table, and was loaded into the PostgreSQL database with the graphical Import/Export tool in pgAdmin. The tables had already been created with their primary and foreign keys, so the order of loading is important.

Load order

Foreign keys make PostgreSQL reject a row whose parent record does not exist. Parent tables were therefore loaded before the tables that refer to them

Import procedure (repeated for each table)

1. In the pgAdmin browser, the table was located under Schemas, then public, then Tables.
2. The table was right-clicked and Import/Export Data was chosen.
3. The Import/Export switch was set to Import.
4. The matching CSV file was selected as the file name and the format was set to csv.
5. On the Options tab, Header was switched on so the first row of column names was skipped, the delimiter was set to a comma, and the encoding was set to UTF8.
6. On the Columns tab, the columns were checked to match the order of the columns in the file.
7. The import was started with OK, and pgAdmin's process notification confirmed that it completed.

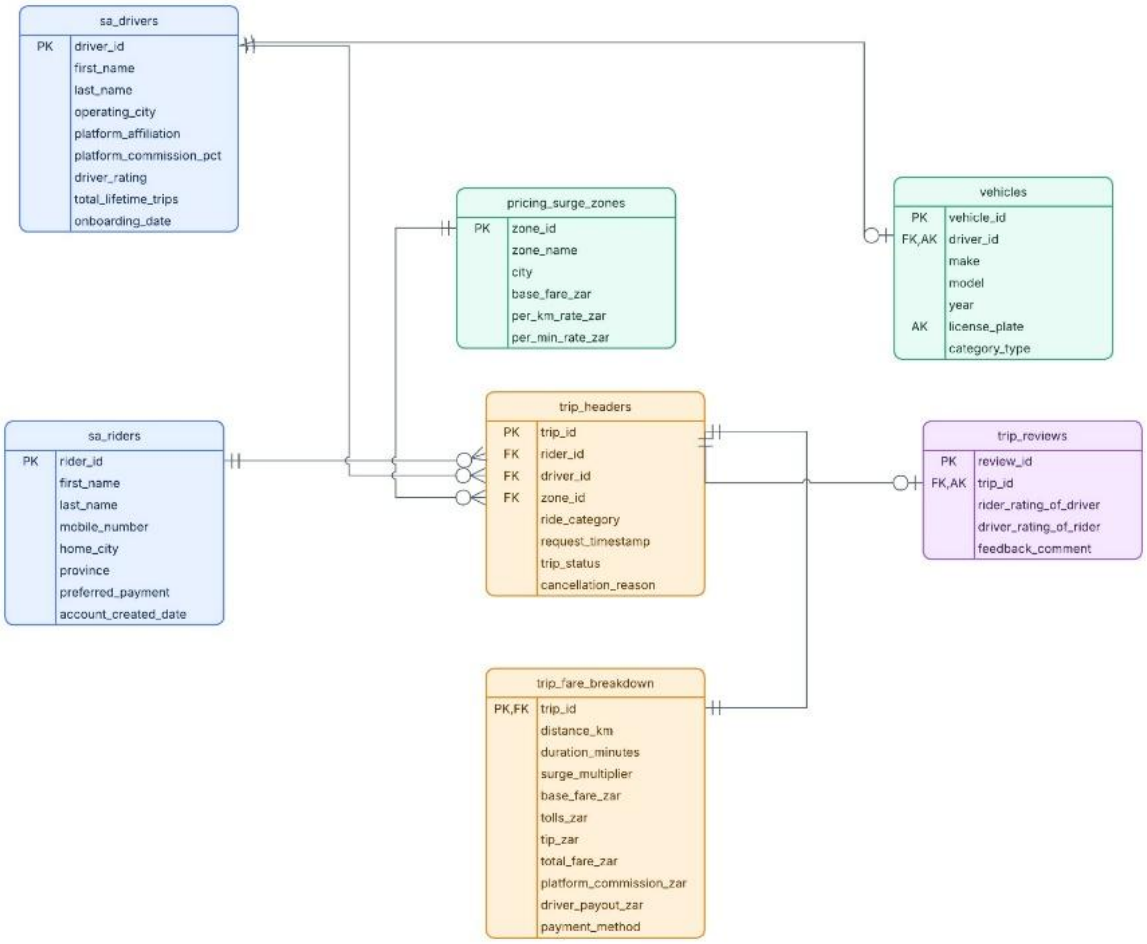
4.6 Power BI Connection and Data Model

All the tables from PostgreSQL are imported into Power BI through Power BI's built-in connector.

Power BI Visualisations Setup

Question	Visual	Fields	Settings applied
Q1: make vs city baseline	Clustered bar chart + a map	**Y axis**: make**.** **X axis**: *gap_vs_baseline*. **Map location**: city	Visual-level filter *driver_count* is at least 5. Clicking a city on the map filters the bars.
Q2: vehicle age vs rating	Line chart (or columns)	**X axis**: *age_bracket*. **Y axis**: *avg_driver_rating*	Sorted age_bracket by *bracket_no*, otherwise the labels sort alphabetically. Added *change_vs_prev_bracket* as a tooltip or label
Q3: market share and longevity	100% stacked bar chart plus a clustered column chart	**Bar chart**: **Y axis:** *operating_city*. **X axis**: *market_share_percentage*. **Legend**: *platform_affiliation*. **Columns chart**: **X axis**: *operating_city*. **Y axis:** *avg_dual_trips* and *avg_exclusive_trips*.	Set the two trip fields to Maximum, since the view repeats each city's value on every platform row. Colour platforms consistently across pages
Q4: tipping by age and category	Clustered column chart	**X axis**: *ride_category*. **Y axis**: *avg_tip_percentage* **Legend**: *vehicle_age_bracket*.	Added a *platform_affiliation* slicer. Use the weighted measure below to combine platforms. Put *total_completed_trips* in the tooltip
Q5: commission vs fleet quality	Scatter chart (bubble)	**X axis**: *avg_commission_pct*. **Y axis**: *avg_driver_rating*. **Legend**: *platform_affiliation*. **Size**: *total_drivers* (we first convert *avg_commission* to a numeric field)	Set the axes to Average. Added *avg_vehicle_age* to the tooltip so the fleet-age side of the question is visible

5. SCHEMA / ERD


sa_drivers
PK driver_id
first_name
last_name
operating_city
platform_affiliation
platform_commission_pct
driver_rating
total_lifetime_trips
onboarding_date
pricing_surge_zones
PK zone_id
zone_name
city
base_fare_zar
per_km_rate_zar
per_min_rate_zar
vehicles
PK vehicle_id
FK,AK driver_id
make
model
year
AK license_plate
category_type
sa_riders
PK rider_id
first_name
last_name
mobile_number
home_city
province
preferred_payment
account_created_date
trip_headers
PK trip_id
FK rider_id
FK driver_id
FK zone_id
ride_category
request_timestamp
trip_status
cancellation_reason
trip_reviews
PK review_id
FK,AK trip_id
rider_rating_of_driver
driver_rating_of_rider
feedback_comment
trip_fare_breakdown
PK,FK trip_id
distance_km
duration_minutes
surge_multiplier
base_fare_zar
tolls_zar
tip_zar
total_fare_zar
platform_commission_zar
driver_payout_zar
payment_method

6. SQL QUERY DEVELOPMENT

QUERY & OUTPUT

QUESTION 1 - Regional Vehicle Make Performance Benchmarking

Query

```
WITH make_city AS (
    SELECT
        d.operating_city      AS city,
        v.make,
        COUNT(*)              AS n,
        SUM(d.driver_rating)  AS rating_sum
    FROM sa_drivers d
    JOIN vehicles v ON v.driver_id = d.driver_id
    WHERE d.driver_rating IS NOT NULL
    GROUP BY d.operating_city, v.make
),
scored AS (
    SELECT
        city,
        make,
        n,
        rating_sum / n                                      AS make_avg,
        SUM(rating_sum) OVER (PARTITION BY city)
          / SUM(n)      OVER (PARTITION BY city)            AS city_baseline
    FROM make_city
)
SELECT
    city,
    make,
    n                                       AS driver_count,
    ROUND(make_avg, 3)                      AS make_avg_rating,
    ROUND(city_baseline, 3)                 AS city_baseline,
    ROUND(make_avg - city_baseline, 3)      AS gap_vs_baseline,
    ROUND(n * (make_avg - city_baseline), 2)   AS rating_points_impact,
    RANK() OVER (PARTITION BY city ORDER BY make_avg DESC) AS rank_in_city,
    CASE WHEN make_avg < city_baseline
         THEN 'BELOW BASELINE' ELSE 'AT/ABOVE BASELINE' END AS status
FROM scored
WHERE n >= 5                -- the baseline above still uses ALL makes
ORDER BY city, rank_in_city;
```

make_city CTE (Common Table Expression):

- Joins SA_DRIVERS and VEHICLES on driver_id (one vehicle per driver, so no rows are duplicated).
- Removes drivers without a rating.
- Groups by city and make, counting the drivers (n) and adding up their ratings (rating_sum).
- It keeps the sum instead of the average so the city baseline can be calculated exactly in the next step.

scored CTE:

- make_avg = rating_sum / n, the average rating for that make in that city.
- city_baseline = the city's total rating points divided by its total drivers. The window functions (SUM ... OVER (PARTITION BY city)) add up all makes in the city, so this is the true average of every driver in the city.

Main SELECT:

- gap_vs_baseline = make average minus city baseline (negative means the make rates below its city).
- rating_points_impact = driver_count x gap, which weights the gap by fleet size so a large make with a small gap can matter more than a tiny make with a big gap.
- rank_in_city uses RANK() so the best-rated make in each city gets 1.
- The CASE statement labels each make BELOW BASELINE or AT/ABOVE BASELINE.
- WHERE n >= 5 removes makes with fewer than five drivers.

Key Takeaways:

- The n >= 5 filter sits in the final SELECT, not inside the CTEs. That way the city baseline still counts every make, while the results only show makes with enough drivers to be reliable.
- Ranks are per city, so a make is only compared with other makes in the same market.
- Ratings belong to drivers, so a make's score reflects the drivers who choose it as well as the car itself.

Output

	city character varying (50)	make character varying (50)	driver_count bigint	make_avg_rating numeric	city_baseline numeric	gap_vs_baseline numeric	rating_points_impact numeric	rank_in_city bigint	status text
1	Cape Town	BMW	13	4.651	4.538	0.113	1.47	1	AT/ABOVE BASELI...
2	Cape Town	Nissan	12	4.613	4.538	0.075	0.90	2	AT/ABOVE BASELI...
3	Cape Town	Hyundai	15	4.569	4.538	0.031	0.46	3	AT/ABOVE BASELI...
4	Cape Town	Suzuki	11	4.564	4.538	0.026	0.29	4	AT/ABOVE BASELI...
5	Cape Town	Renault	14	4.532	4.538	-0.006	-0.08	5	BELOW BASELINE
6	Cape Town	Toyota	29	4.500	4.538	-0.038	-1.09	6	BELOW BASELINE
7	Cape Town	Kia	11	4.476	4.538	-0.061	-0.67	7	BELOW BASELINE
8	Cape Town	Volkswagen	17	4.463	4.538	-0.075	-1.27	8	BELOW BASELINE
9	Durban	Suzuki	12	4.636	4.554	0.082	0.99	1	AT/ABOVE BASELI...
10	Durban	Volkswagen	8	4.629	4.554	0.075	0.60	2	AT/ABOVE BASELI...

Total rows: 56 Query complete 00:00:00.113

Interpretation:

- A positive gap_vs_baseline means drivers of that make are rated above their city average; a negative gap means below.
- A large negative rating_points_impact flags a make with many drivers that is pulling the city average down.
- The query shows association only. It cannot prove that a make causes better or worse ratings.

QUESTION 2 - Asset Age Cut-off and Passenger Satisfaction

Query

```
WITH aged AS (
    SELECT d.driver_rating,
        CASE
            WHEN EXTRACT(YEAR FROM CURRENT_DATE)::INT - v.year <= 2 THEN 1
            WHEN EXTRACT(YEAR FROM CURRENT_DATE)::INT - v.year <= 5 THEN 2
            WHEN EXTRACT(YEAR FROM CURRENT_DATE)::INT - v.year <= 8 THEN 3
            ELSE 4
        END AS bracket_no
    FROM vehicles   v
    JOIN sa_drivers d ON d.driver_id = v.driver_id
    WHERE d.driver_rating IS NOT NULL
      AND v.year IS NOT NULL
),
stats AS (
    SELECT
        bracket_no,
        COUNT(*)           AS driver_count,
        SUM(driver_rating) AS rating_sum,
        MIN(driver_rating) AS min_rating,
        MAX(driver_rating) AS max_rating
    FROM aged
    GROUP BY bracket_no
) SELECT
    CASE bracket_no
        WHEN 1 THEN '0-2 years'
        WHEN 2 THEN '3-5 years'
        WHEN 3 THEN '6-8 years'
        ELSE        '9+ years'
    END                                                    AS age_bracket,
    driver_count,
    ROUND(rating_sum / driver_count, 3)                    AS avg_driver_rating,
    ROUND(rating_sum / driver_count
          - LAG(rating_sum / driver_count) OVER (ORDER BY bracket_no), 3)
                                                           AS change_vs_prev_bracket,
    ROUND(rating_sum / driver_count
          - SUM(rating_sum) OVER () / SUM(driver_count) OVER (), 3)
                                                           AS gap_vs_fleet_avg,
    min_rating,
    max_rating
FROM stats
ORDER BY bracket_no;
```

aged CTE (Common Table Expression):

- Joins VEHICLES and SA_DRIVERS on driver_id.
- Calculates vehicle age as the current year minus the vehicle's model year. Using CURRENT_DATE means the ages stay correct without editing the query each year.
- Assigns each driver a bracket number: 1 = 0-2 years, 2 = 3-5, 3 = 6-8, 4 = 9 or more.
- Removes rows with a missing rating or model year.

stats CTE:

- Groups by bracket_no.
- Calculates the driver count, the sum of ratings, and the lowest and highest rating in each bracket.

Main SELECT:

- The CASE statement turns bracket numbers into readable labels.
- avg_driver_rating = rating_sum / driver_count.

- change_vs_prev_bracket uses LAG() ordered by bracket_no to subtract the previous bracket's average. This is the step change; it is empty for the first bracket.
- gap_vs_fleet_avg subtracts the average of the whole fleet (calculated with SUM() OVER ()) from each bracket's average.
- min_rating and max_rating show how spread out the ratings are inside each bracket.

Key Takeaways:

- Brackets are stored as numbers (1 to 4) and labelled only at the end, so the output sorts by age instead of alphabetically.
- Because the fleet average is built from sums and counts, it is weighted correctly instead of being an average of averages.
- The brackets are 0-2, 3-5, 6-8 and 9+. The tipping query (Question 4) uses different cut-offs (0-3, 4-7, 8+), so the two results should not be compared bracket for bracket.

Output

	age_bracket text	driver_count bigint	avg_driver_rating numeric	change_vs_prev_bracket numeric	gap_vs_fleet_avg numeric	min_rating numeric	max_rating numeric
1	0-2 years	164	4.543	[null]	-0.006	4.11	4.99
2	3-5 years	225	4.555	0.013	0.007	4.10	4.99
3	6-8 years	258	4.568	0.013	0.019	4.10	5.00
4	9+ years	153	4.514	-0.054	-0.035	4.11	4.98

Total rows: 4 Query complete 00:00:00.074

Interpretation:

- The first bracket with a clearly negative change_vs_prev_bracket marks the age cut-off where satisfaction starts to fall.
- Small changes between brackets would suggest vehicle age is not a major driver of passenger satisfaction.
- A wide min-to-max range in a bracket means individual driver behaviour matters more than vehicle age in that group.

QUESTION 3 - Platform market share & dual vs. exclusive driver longevity

Query

```sql
WITH market_share AS (
    SELECT
        operating_city,
        platform_affiliation,
        COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (PARTITION BY operating_city) AS market_share_percentage
    FROM sa_drivers
    GROUP BY operating_city, platform_affiliation
),
city_averages AS (
    SELECT
        operating_city,
        AVG(CASE WHEN platform_affiliation = 'Dual-Platform (Both)' THEN total_lifetime_trips END) AS avg_dual_trips,
        AVG(CASE WHEN platform_affiliation = 'Uber' THEN total_lifetime_trips END) AS avg_uber_trips,
        AVG(CASE WHEN platform_affiliation = 'Bolt' THEN total_lifetime_trips END) AS avg_bolt_trips,
        AVG(CASE WHEN platform_affiliation != 'Dual-Platform (Both)' THEN total_lifetime_trips END) AS avg_exclusive_trips
    FROM sa_drivers
    GROUP BY operating_city
)
SELECT
    ms.operating_city,
    ms.platform_affiliation,
    ROUND(ms.market_share_percentage, 2) AS market_share_percentage,
    ROUND(ca.avg_dual_trips, 2) AS avg_dual_trips,
    ROUND(ca.avg_uber_trips, 2) AS avg_uber_trips,
    ROUND(ca.avg_bolt_trips, 2) AS avg_bolt_trips,
    ROUND(ca.avg_exclusive_trips, 2) AS avg_exclusive_trips,
    CASE
        WHEN ca.avg_dual_trips > ca.avg_exclusive_trips THEN 'Dual Outperforms Exclusive'
        WHEN ca.avg_dual_trips < ca.avg_exclusive_trips THEN 'Exclusive Outperforms Dual'
        ELSE 'Equal Longevity'
    END AS longevity_comparison
FROM market_share ms
JOIN city_averages ca ON ms.operating_city = ca.operating_city
ORDER BY ms.operating_city, ms.platform_affiliation;
```

market_share CTE (Common Table Expression):

- Reads only the SA_DRIVERS table.
- Counts the drivers for each city and platform affiliation.
- Divides that count by the city's total driver count, which comes from a window function (SUM(COUNT(*)) OVER (PARTITION BY city)). Multiplying by 100.0 forces decimal division and gives a percentage.

city_averages CTE:

- Calculates the average total_lifetime_trips per city for each group of drivers.
- Each AVG(CASE WHEN ... THEN trips END) only averages the drivers that match the condition, because rows that do not match become NULL and AVG ignores NULL.
- avg_exclusive_trips covers every driver who is not dual-platform, meaning Uber and Bolt combined.

Main SELECT:

- Joins the two CTEs on operating_city, so each platform row also carries its city's longevity averages.
- Rounds the percentages and averages to two decimals.
- The CASE statement compares dual and exclusive averages and labels the city: Dual Outperforms Exclusive, Exclusive Outperforms Dual, or Equal Longevity.

Key Takeaways:

- total_lifetime_trips is used as the measure of driver longevity.
- The longevity averages are per city, so they repeat on every platform row of that city. The market share column is the only value that changes by platform.
- The query depends on the exact text 'Dual-Platform (Both)', so a spelling change in the data would silently produce NULL averages.
- Lifetime trips also grows with time on the platform. A dual-platform group made up of older drivers would look better for that reason alone. Dividing trips by months since onboarding_date would remove this effect.

Output

	operating_city character varying (50)	platform_affiliation character varying (50)	market_share_percentage numeric	avg_dual_trips numeric	avg_uber_trips numeric	avg_bolt_trips numeric	avg_exclusive_trips numeric	longevity_comparison text
1	Cape Town	Bolt	37.70	3097.15	3828.17	3404.93	3587.81	Exclusive Outperforms D…
2	Cape Town	Dual-Platform (Both)	33.61	3097.15	3828.17	3404.93	3587.81	Exclusive Outperforms D…
3	Cape Town	Uber	28.69	3097.15	3828.17	3404.93	3587.81	Exclusive Outperforms D…
4	Durban	Bolt	34.23	3567.09	3031.95	3714.74	3360.38	Dual Outperforms Exclusi…
5	Durban	Dual-Platform (Both)	28.83	3567.09	3031.95	3714.74	3360.38	Dual Outperforms Exclusi…
6	Durban	Uber	36.94	3567.09	3031.95	3714.74	3360.38	Dual Outperforms Exclusi…
7	Gqeberha	Bolt	31.53	2850.03	3817.79	3372.86	3604.47	Exclusive Outperforms D…
8	Gqeberha	Dual-Platform (Both)	34.23	2850.03	3817.79	3372.86	3604.47	Exclusive Outperforms D…
9	Gqeberha	Uber	34.23	2850.03	3817.79	3372.86	3604.47	Exclusive Outperforms D…
10	Johannesburg	Bolt	31.63	3381.97	4167.85	3965.42	4069.80	Exclusive Outperforms D…

Total rows: 21 Query complete 00:00:00.080

Interpretation:

- Market share shows which platform dominates each city and where a platform is weak.
- If longevity_comparison reads Dual Outperforms Exclusive, drivers who work both apps accumulate more trips, which suggests multi-homing keeps drivers active.
- If it reads Exclusive Outperforms Dual, loyalty to a single platform goes together with more trips.
- Cities where the verdict differs show that the effect is not the same everywhere.

QUESTION 4 - Vehicle Age & Category Influence on Rider Tipping Behaviour

Query

```sql
WITH trip_tips_derived AS (
    SELECT
        th.trip_id,
        th.driver_id,
        th.ride_category,
        EXTRACT(YEAR FROM CURRENT_DATE) - v.year AS vehicle_age_years,
        CASE
            WHEN (EXTRACT(YEAR FROM CURRENT_DATE) - v.year) <= 3 THEN '1. 0-3 Years (New)'
            WHEN (EXTRACT(YEAR FROM CURRENT_DATE) - v.year) BETWEEN 4 AND 7 THEN '2. 4-7 Years (Mid-Age)'
            ELSE '3. 8+ Years (Aging)'
        END AS vehicle_age_bracket,
        tf.tip_zar,
        tf.total_fare_zar,
        (tf.tip_zar / NULLIF(tf.total_fare_zar, 0)) * 100 AS tip_pct
    FROM trip_headers th
    INNER JOIN trip_fare_breakdown tf ON th.trip_id = tf.trip_id
    INNER JOIN vehicles v ON th.driver_id = v.driver_id
    WHERE th.trip_status = 'COMPLETED'
      AND tf.total_fare_zar > 0
)
SELECT
    d.platform_affiliation,
    tt.ride_category,
    tt.vehicle_age_bracket,
    COUNT(tt.trip_id) AS total_completed_trips,
    ROUND(CAST(AVG(tt.tip_zar) AS NUMERIC), 2) AS avg_tip_zar,
    ROUND(CAST(AVG(tt.tip_pct) AS NUMERIC), 2) AS avg_tip_percentage
FROM trip_tips_derived tt
INNER JOIN sa_drivers d ON tt.driver_id = d.driver_id
GROUP BY
    d.platform_affiliation,
    tt.ride_category,
    tt.vehicle_age_bracket
ORDER BY
    d.platform_affiliation,
    tt.ride_category,
    tt.vehicle_age_bracket;
```

trip_tips_derived CTE (Common Table Expression):

- Joins TRIP_HEADERS, TRIP_FARE_BREAKDOWN and VEHICLES, so every trip is linked to its fare and to the car that drove it.
- Keeps only COMPLETED trips with a total fare above zero, since cancelled trips have no fare to tip on.
- Calculates the vehicle age and places it in one of three brackets: 0-3 years (new), 4-7 years (mid-age) and 8 or more (aging).
- Calculates tip_pct = tip / total fare x 100. NULLIF(total_fare_zar, 0) turns a zero fare into NULL, which prevents a divide-by-zero error.

Main SELECT:

- Joins the CTE to SA_DRIVERS to bring in the platform.
- Groups by platform, ride category and age bracket.
- Reports the number of completed trips, the average tip in rand and the average tip percentage.
- CAST(AS NUMERIC) is a safeguard: PostgreSQL's ROUND(value, 2) fails on double precision values, which some expressions such as EXTRACT return in older versions.

- Orders by platform, category and bracket.

Key Takeaways:

- Only completed trips are counted, so the averages describe trips where a tip was possible.
- The bracket labels start with 1., 2. and 3. so alphabetical sorting also puts them in age order.
- The vehicle join does not duplicate trips because each driver has exactly one vehicle (vehicles.driver_id is UNIQUE).
- tip_pct is calculated per trip and then averaged, so a short cheap trip weighs the same as a long expensive one. A trips-weighted alternative would be total tips divided by total fares.
- Trips with a tip of zero count in the averages, so the figures reflect all riders, not just those who tipped.

Output

	platform_affiliation character varying (50)	ride_category character varying (60)	vehicle_age_bracket text	total_completed_trips bigint	avg_tip_zar numeric	avg_tip_percentage numeric
1	Bolt	Budget (Uber Go/Bolt Go)	1. 0-3 Years (New)	907	15.44	4.68
2	Bolt	Budget (Uber Go/Bolt Go)	2. 4-7 Years (Mid-Ag...	1065	17.17	5.13
3	Bolt	Budget (Uber Go/Bolt Go)	3. 8+ Years (Aging)	763	16.09	4.69
4	Bolt	Comfort (Uber Comfort)	1. 0-3 Years (New)	869	16.08	5.09
5	Bolt	Comfort (Uber Comfort)	2. 4-7 Years (Mid-Ag...	1025	15.52	4.79
6	Bolt	Comfort (Uber Comfort)	3. 8+ Years (Aging)	649	16.06	4.84
7	Bolt	Large Capacity (UberXL/Bolt ...	1. 0-3 Years (New)	829	15.68	5.12
8	Bolt	Large Capacity (UberXL/Bolt ...	2. 4-7 Years (Mid-Ag...	1076	15.27	4.79
9	Bolt	Large Capacity (UberXL/Bolt ...	3. 8+ Years (Aging)	744	15.87	4.81
10	Bolt	Standard (UberX/Bolt)	1. 0-3 Years (New)	923	15.52	5.01
11	Bolt	Standard (UberX/Bolt)	2. 4-7 Years (Mid-Ag...	1096	16.22	4.87
12	Bolt	Standard (UberX/Bolt)	3. 8+ Years (Aging)	735	15.90	4.64
13	Dual-Platform (Both)	Budget (Uber Go/Bolt Go)	1. 0-3 Years (New)	719	15.54	4.70

Total rows: 36 | Query complete 00:00:00.116

Interpretation:

- Higher average tip percentages in the 0-3 year bracket would suggest riders reward newer cars.
- Differences between ride categories show whether premium services attract more generous tipping.

QUESTION 5 - Platform Commission vs. Fleet Quality

Query

```sql
SELECT
    d.platform_affiliation,
    ROUND(AVG(d.platform_commission_pct) * 100, 1) || '%' AS avg_commission_pct,
    COUNT(DISTINCT d.driver_id) AS total_drivers,
    ROUND(AVG(2026 - v.year), 1) AS avg_vehicle_age,
    ROUND(AVG(d.driver_rating), 2) AS avg_driver_rating
FROM sa_drivers d
JOIN vehicles v ON d.driver_id = v.driver_id
WHERE d.driver_rating IS NOT NULL
GROUP BY d.platform_affiliation
ORDER BY avg_commission_pct DESC;
```

FROM and JOIN:

- Joins SA_DRIVERS and VEHICLES on driver_id, so each driver is paired with their vehicle.
- WHERE removes drivers with no rating, so the averages are not distorted by missing values.

SELECT and GROUP BY (one row per platform):

- avg_commission_pct = the average commission fraction x 100, rounded to one decimal, with a % sign added as text.
- total_drivers = COUNT(DISTINCT driver_id), the number of drivers behind the averages.
- avg_vehicle_age = the average of (2026 - model year), the fleet's average age in years.
- avg_driver_rating = the average rating, rounded to two decimals.

Key Takeaways:

- The result is a compact platform scorecard: price (commission), asset quality (vehicle age) and service quality (rating) in one place.
- The year 2026 is typed into the query. It must be updated each year; CURRENT_DATE, as used in Questions 2 and 4, would keep it correct automatically.
- Adding the % sign turns the commission into text. ORDER BY avg_commission_pct DESC then sorts alphabetically instead of numerically, so a value such as 9.5% can appear above 25.0%. Sorting on the numeric average, or leaving the % out of the query, fixes this.
- Dual-platform drivers form their own group, so they are compared as a third platform alongside Uber and Bolt.

Output

	platform_affiliation character varying (50)	avg_commission_pct text	total_drivers bigint	avg_vehicle_age numeric	avg_driver_rating numeric
1	Uber	25.0%	280	5.5	4.53
2	Dual-Platform (Both)	22.0%	259	5.7	4.56
3	Bolt	20.0%	261	5.3	4.55

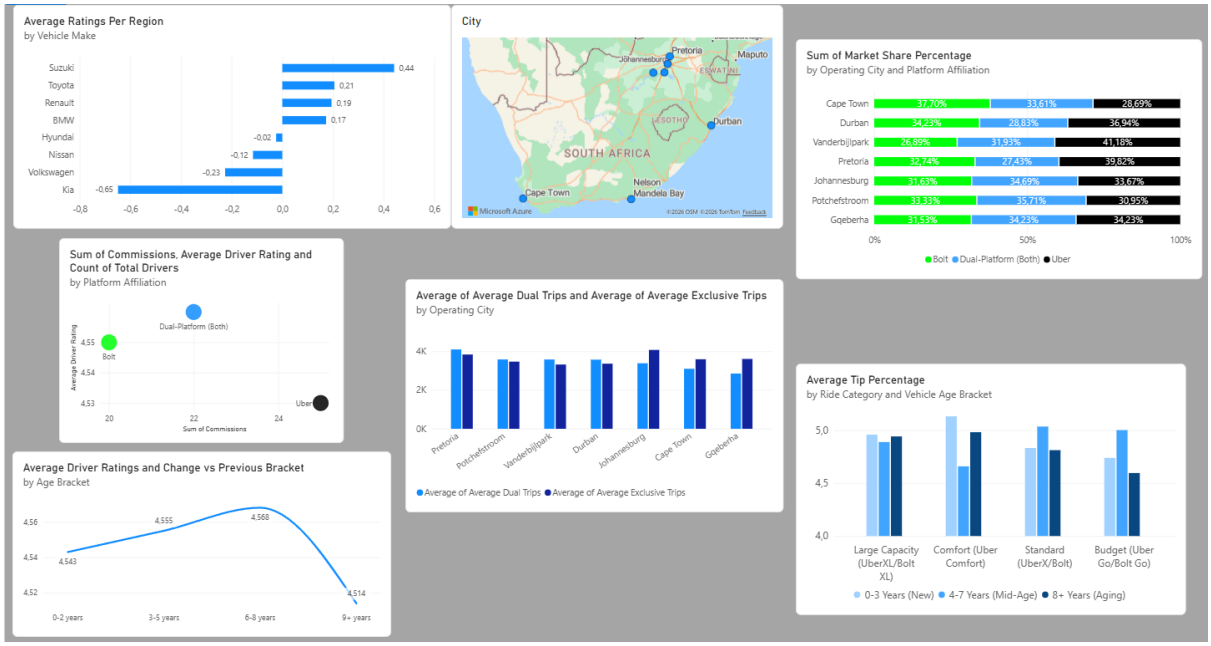
Total rows: 3 Query complete 00:00:00.096

Interpretation:

- A higher commission paired with a newer fleet and higher ratings would suggest the platform trades a bigger cut for quality.
- A higher commission with an older fleet and lower ratings would suggest drivers receive less value for what they give up.
- With only a few platform rows, the pattern is descriptive. It cannot show that commission causes the difference in fleet quality.

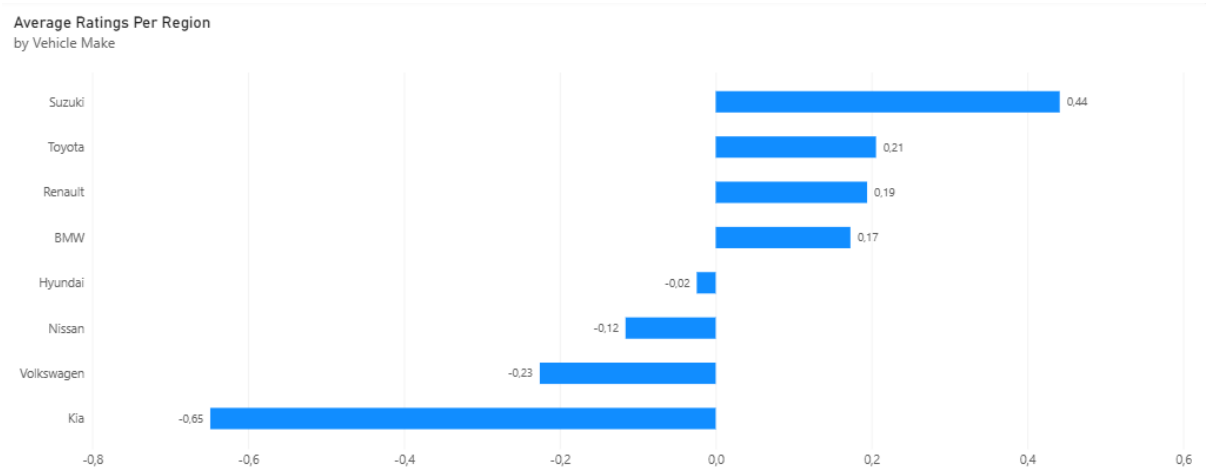
7. ANALYSIS AND REPORTING

Summary Dashboard

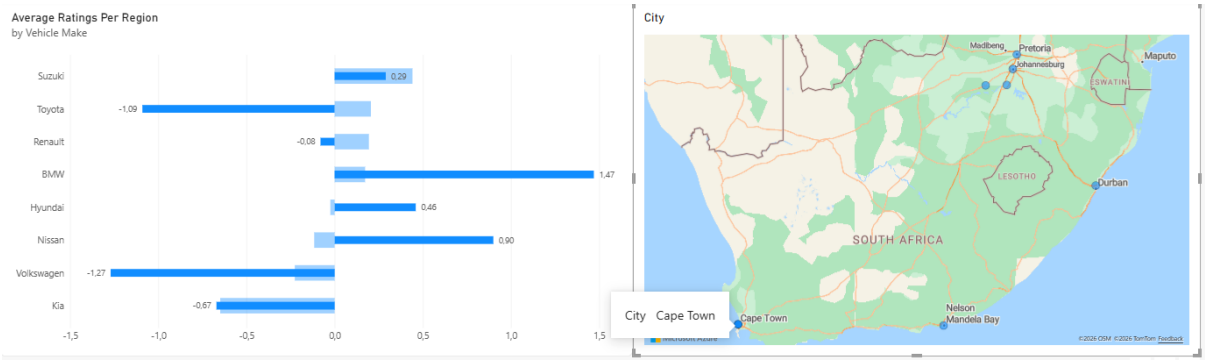


Question 1:

Countywide Baseline



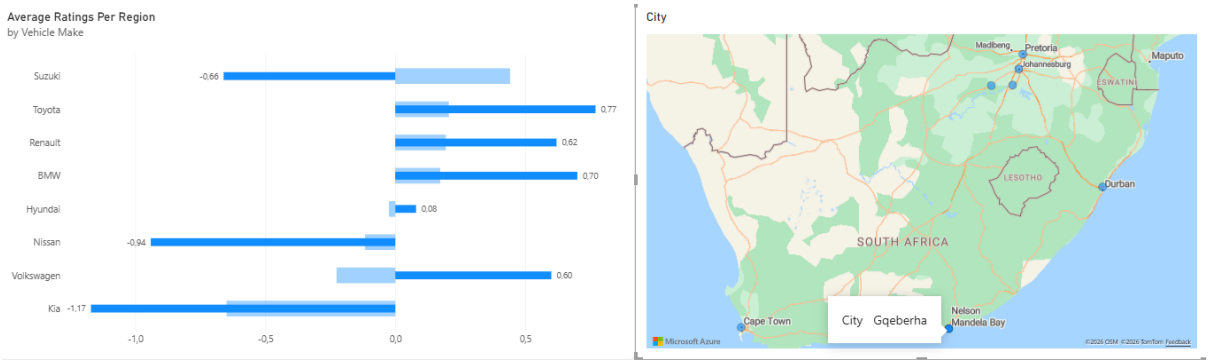
Cape Town vs. Countrywide Baseline:



BMW (+1.47) and Nissan (+0.90) lead, and Volkswagen (−1.27) and Toyota (−1.09) trail.

- Nissan swings the most: slightly negative overall (−0.12) but strongly positive here.
- Toyota flips from above to well below baseline, about 1.3 points worse than its overall result.
- Kia (−0.67) matches its overall result (−0.65), so it is weak here for the same reason as everywhere else.

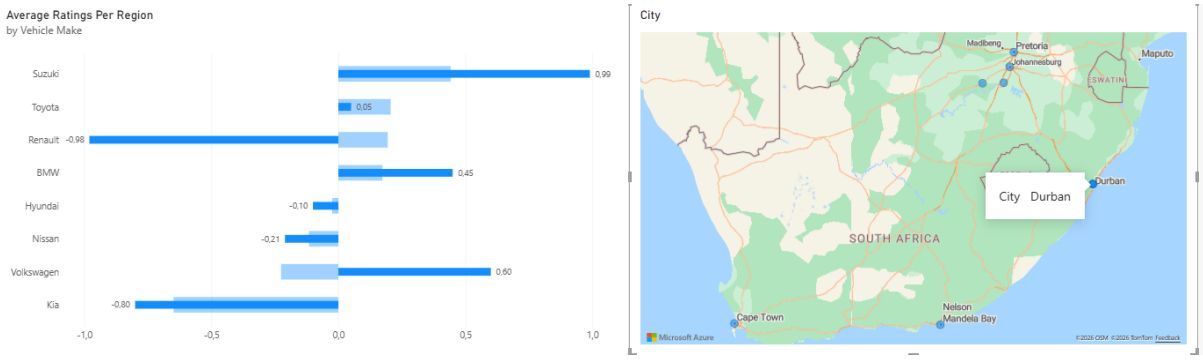
Gqeberha vs. Countrywide Baseline:



Toyota, BMW, Renault and Volkswagen are all around +0.6 to +0.8, and Kia (−1.17), Nissan (−0.94) and Suzuki (−0.66) trail.

- Suzuki is the overall leader (+0.44) but is below baseline here, a swing of about 1.1 points.
- Volkswagen turns from below baseline overall to +0.60 here.

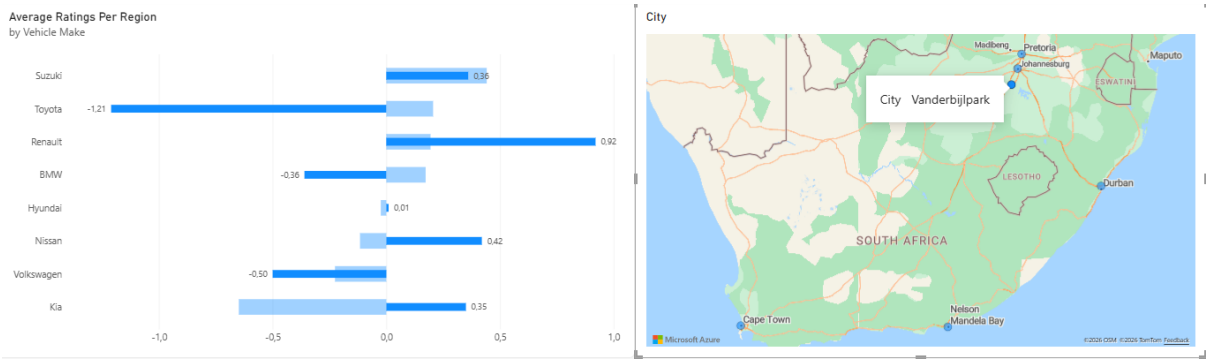
Durban vs. Countrywide Baseline:



Suzuki (+0.99) is the clear leader, and Renault (−0.98) and Kia (−0.80) trail.

- Suzuki's lead is bigger here than overall.
- Renault flips from slightly positive overall (+0.19) to the worst make, a swing of about 1.2 points.
- Volkswagen is again above baseline (+0.60), as in Gqeberha.
- Toyota, Hyundai and Nissan are close to baseline.

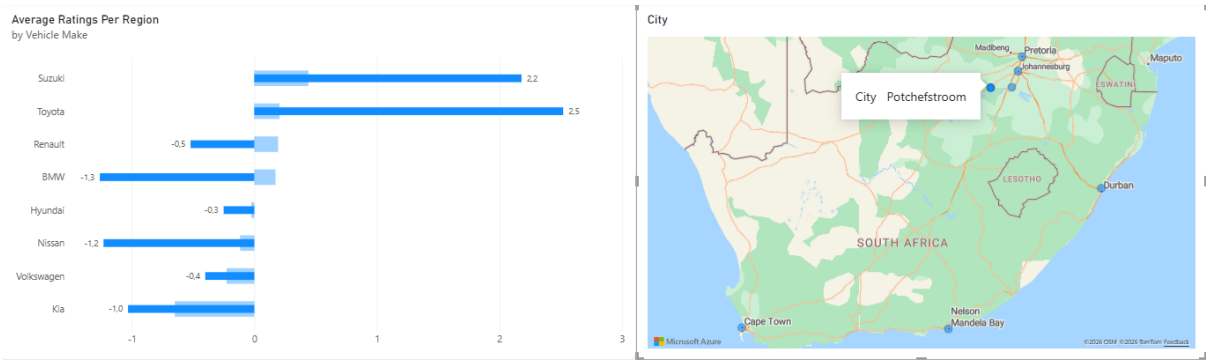
Vanderbijlpark vs. Countrywide Baseline:



Renault (+0.92) leads, and Toyota (−1.21) trails.

- Kia is above baseline (+0.35), a swing of about 1.0 point from its overall −0.65.
- Toyota is about 1.4 points worse than its overall result, the biggest negative deviation in this city.
- Hyundai is almost exactly on baseline (+0.01).

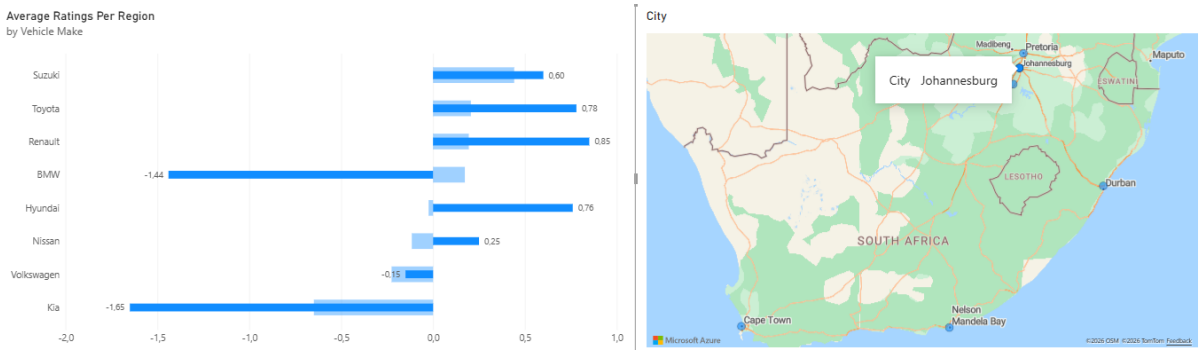
Potchefstroom vs. Countrywide Baseline:



Toyota (+2.5) and Suzuki (+2.2) are far above everyone else, and all six other makes are negative (BMW −1.3, Nissan −1.2, Kia −1.0).

- This is the most extreme panel, so treat it with caution (see the checks below).

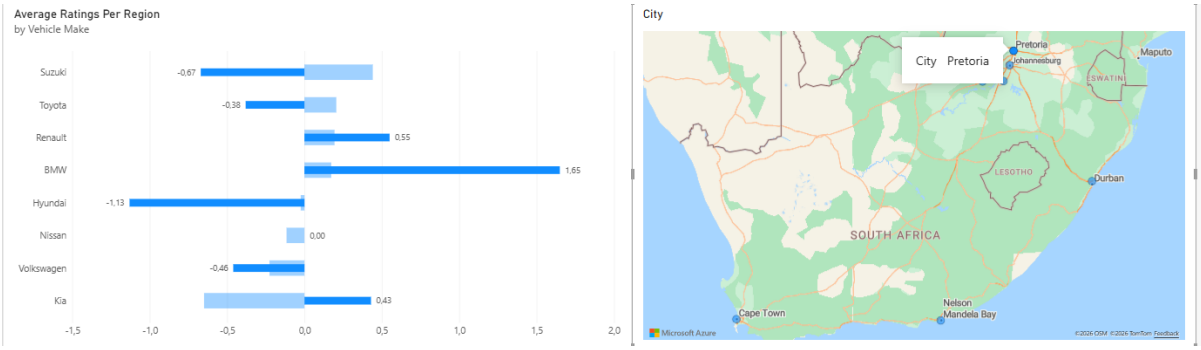
Johannesburg vs Countrywide Baseline:



Renault (+0.85), Toyota (+0.78) and Hyundai (+0.76) lead, and Kia (−1.65) and BMW (−1.44) trail badly.

- Kia has its worst result of any city here.
- BMW is positive overall but about 1.6 points lower here, the biggest negative deviation in this city.
- Hyundai moves from flat overall to +0.76.

Pretoria vs. Countrywide Baseline:



BMW (+1.65) leads, and Hyundai (−1.13) and Suzuki (−0.67) trail.

- This is the opposite of Johannesburg for BMW, despite the two cities being close together.
- Suzuki (−0.67) is below baseline here, as in Gqeberha.
- Kia is positive (+0.43), the second city where it beats its overall result.
- Nissan sits at 0.00.

Summary Table

Make	All regions	Cape Town	Gqeberha	Durban	Vanderbijlpark	Potchefstroom	Johannesburg	Pretoria
Suzuki	+0.44	+0.29	−0.66	+0.99	+0.36	+2.2	+0.60	−0.67
Toyota	+0.21	−1.09	+0.77	+0.05	−1.21	+2.5	+0.78	−0.38
Renault	+0.19	−0.08	+0.62	−0.98	+0.92	−0.5	+0.85	+0.55
BMW	+0.17	+1.47	+0.70	+0.45	−0.36	−1.3	−1.44	+1.65
Hyundai	−0.02	+0.46	+0.08	−0.10	+0.01	−0.3	+0.76	−1.13
Nissan	−0.12	+0.90	−0.94	−0.21	+0.42	−1.2	+0.25	0.00
Volkswagen	−0.23	−1.27	+0.60	+0.60	−0.50	−0.4	−0.15	−0.46
Kia	−0.65	−0.67	−1.17	−0.80	+0.35	−1.0	−1.65	+0.43

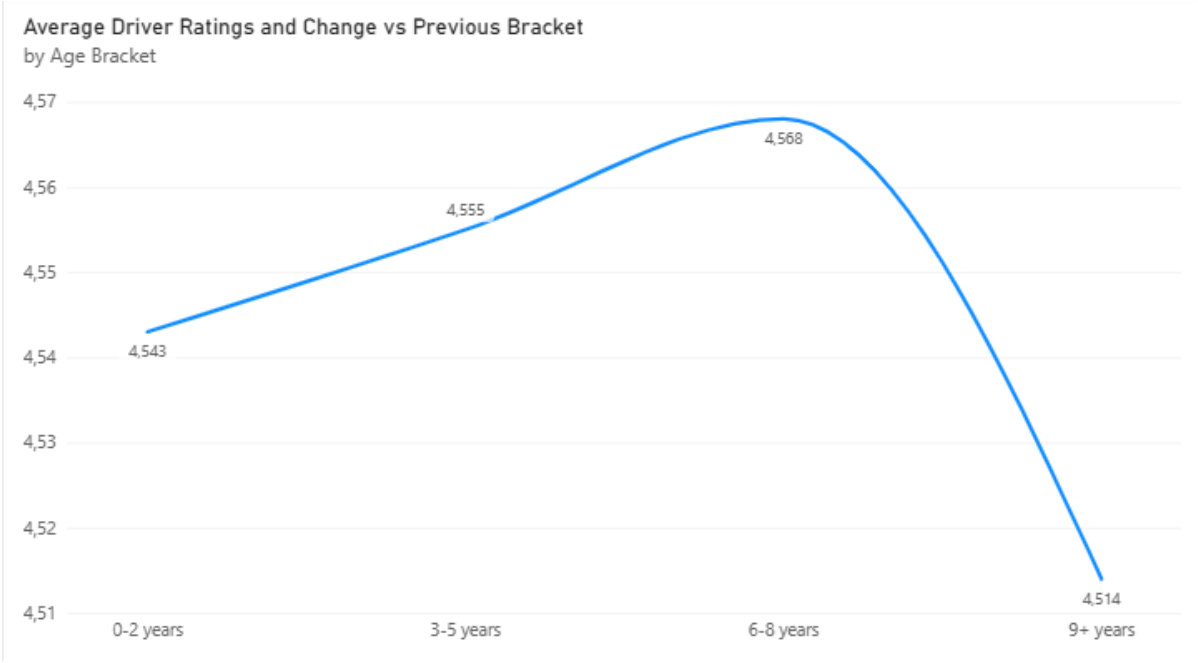
Patterns across regions

- **Suzuki** is above baseline in five of seven cities, and Durban and Potchefstroom are its two strongest.
- **Kia** is below baseline in five of seven cities, and Vanderbijlpark and Pretoria are the only exceptions.
- **BMW** is the most volatile make, ranging from +1.65 in Pretoria to −1.44 in Johannesburg. Cape Town and Pretoria are strongly positive, and Johannesburg and Potchefstroom are strongly negative.
- **Toyota** ranges from +2.5 in Potchefstroom to −1.21 in Vanderbijlpark. It is positive in four cities and negative in three.
- **Hyundai** is near baseline in most cities, with the biggest swings in Pretoria (−1.13) and Johannesburg (+0.76).

Conclusion and Action:

Makes rank differently in every city, so a single national standard hides local underperformance. Kia and Volkswagen fall below their city baseline in five of seven cities, while Toyota, BMW, Nissan, Renault and Hyundai swing between above and below. Therefore, InDrive should adopt city-specific onboarding criteria and place Kia (Johannesburg, Gqeberha, Potchefstroom, Durban and Cape Town) and Volkswagen (starting with Cape Town, where it is worst) on a watch-list. Upgrade subsidies or coaching should come first, and a make should be restricted in a city only if it stays below baseline over several quarters.

Question 2:



- **Ratings rise slowly with age until 6-8 years.** The two steps are +0.012 and +0.013, so the first three brackets are essentially flat. The 6-8 year bracket is the best at 4.568.
- **The cut-off is at 9+ years.** The step change is −0.054, about four times larger than the earlier steps and in the opposite direction. This is the age at which satisfaction starts to fall, which is what the question asks for.
- **The oldest bracket is also the lowest.** At 4.514 it sits 0.029 below the newest cars (4.543), so the oldest cars rate worse than the newest ones.

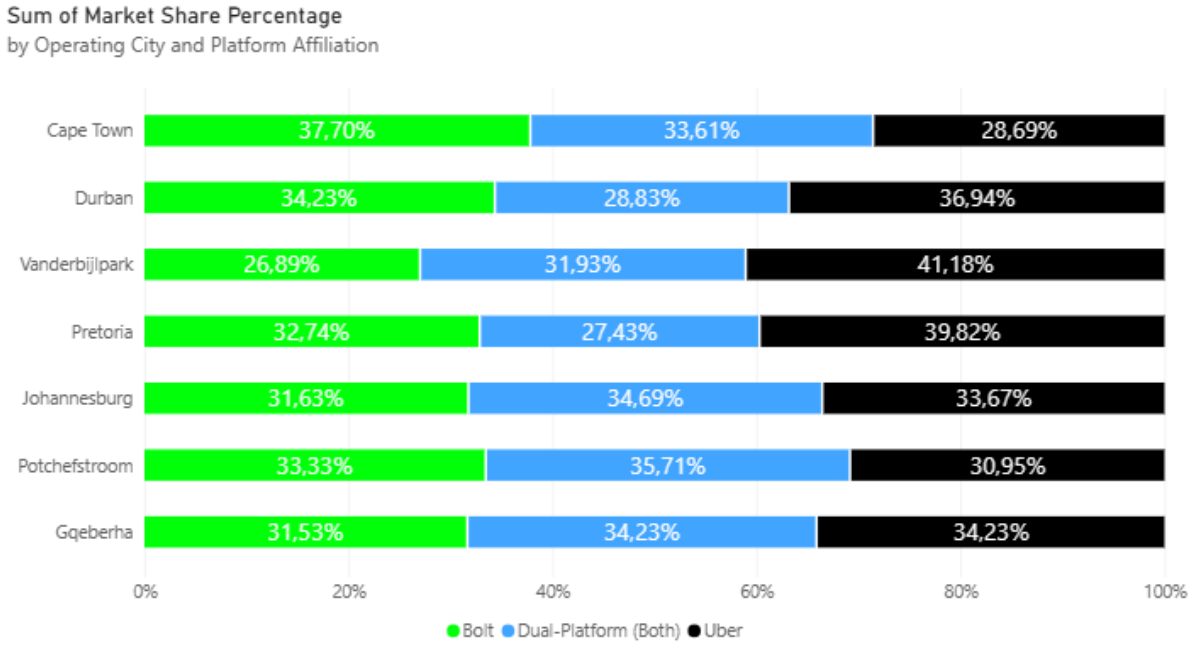
Interpretation

- Cars up to 8 years old are rated about the same, and the mid-age cars do slightly better than the newest. Newer is not better in this data.
- The slight rise up to 6-8 years may reflect more experienced drivers owning slightly older cars, rather than anything about the cars. Ratings belong to drivers, so this shows association only.

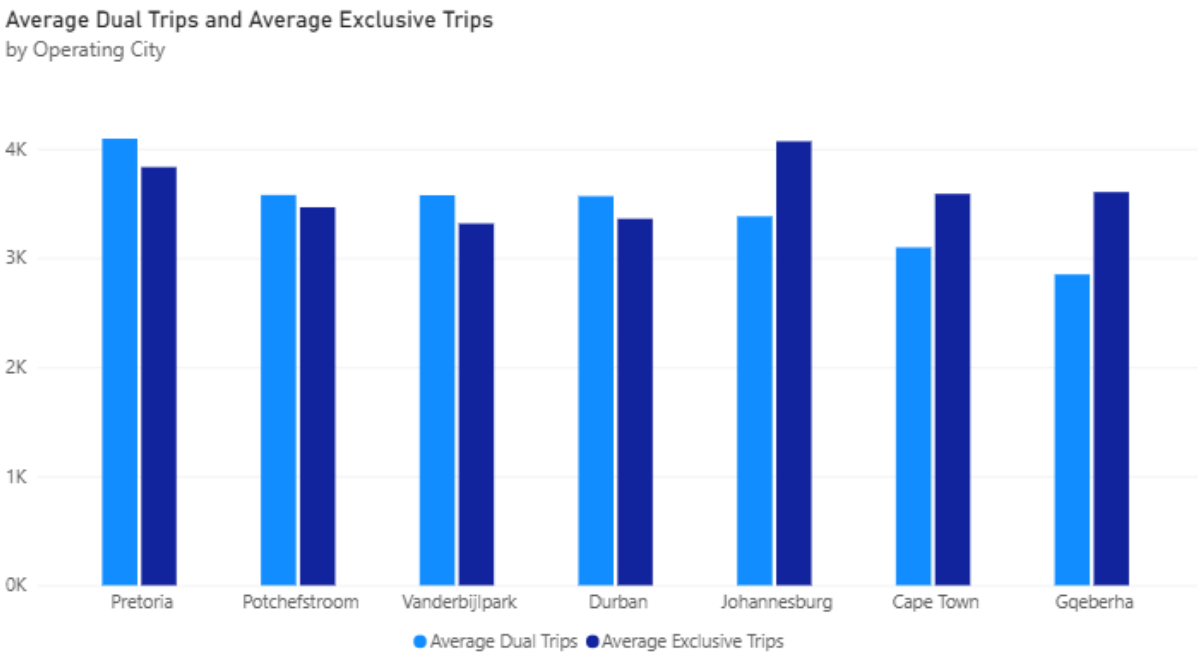
Conclusion & Action:

Ratings are flat from 0 to 8 years and fall only at 9+ years (4.514, against 4.543 to 4.568 in the younger brackets), so the data supports an age threshold, although the effect is small at 0.054 points. Therefore, InDrive should cap vehicle age at 8 years for premium tiers and offer trade-in discounts and renewal incentives to drivers whose cars are 7 to 8 years old, without tightening the rules for newer cars.

Question 3:



- **Uber** leads in Durban, Vanderbijlpark and Pretoria. Its strongest city is Vanderbijlpark (41.18%, a lead of about 9 points over Dual).
- **Bolt** leads only in Cape Town (37.70%). It is weakest in Vanderbijlpark (26.89%).
- **Dual-platform drivers** are the largest group in Johannesburg, Potchefstroom and Gqeberha (tied there), and the smallest in Pretoria (27.43%) and Durban (28.83%).
- **Gaps are small.** In Johannesburg the leader is only one point ahead, and Gqeberha is an exact tie, so those rankings should not be over-read.



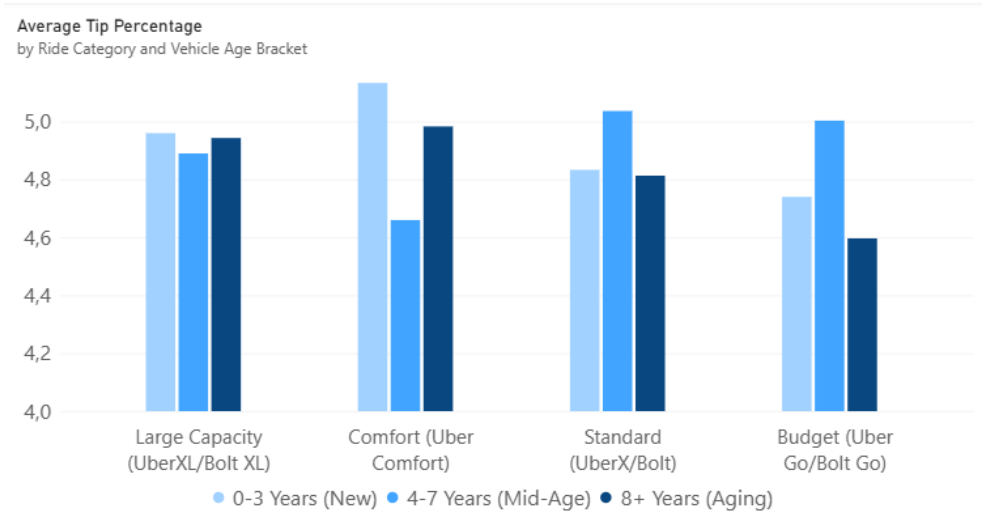
Interpretation

- Dual-platform drivers win in four cities, but only by about 0.1K to 0.3K trips.
- Exclusive drivers win in three cities by 0.5K to 0.75K trips, which is a much larger margin.
- Gqeberha has the biggest gap, and its dual group also has the lowest average of any city.
- Johannesburg's exclusive drivers have the highest exclusive average.

Conclusion & Action

The claim that dual drivers last longer is true Johannesburg, Cape Town and Gqeberha and false in Pretoria, Durban, Vanderbijlpark and Potchefstroom. Therefore, InDrive should target dual-platform drivers with lower commission and flexible dispatch in Pretoria, Durban, Vanderbijlpark and Potchefstroom, as there is clear disparity

Question 4:



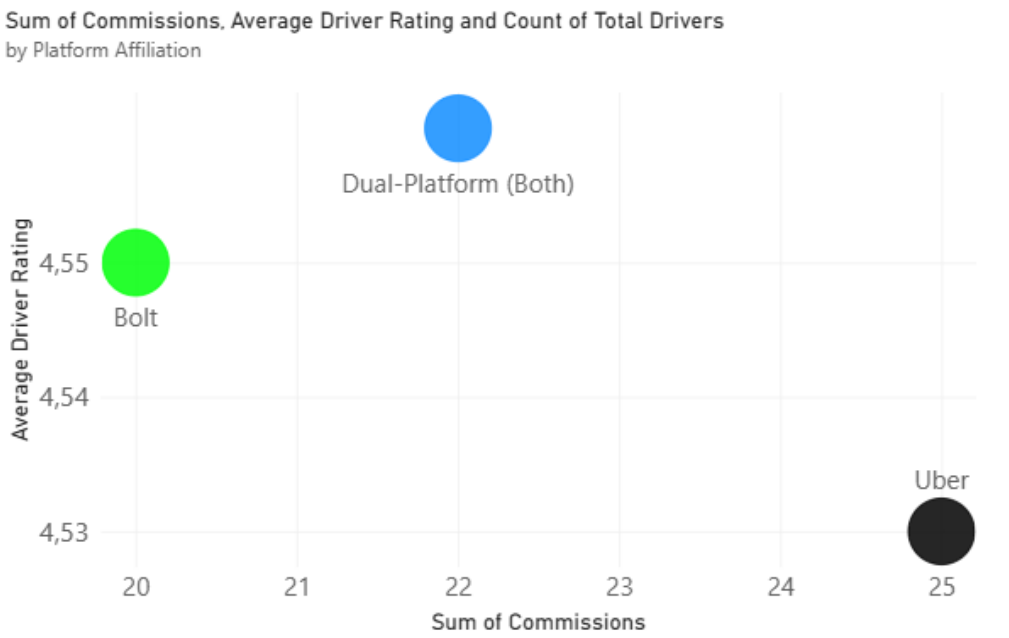
Interpretation

1. **Newer cars do not reliably earn bigger tips.** New cars win in two categories (Large Capacity, Comfort) and mid-age cars win in the other two. A rule such as "riders reward newer vehicles" is not supported.
2. **Aging cars are not penalised much.** They are the lowest bracket in only two of four categories (Standard and Budget), and in Budget they are ~0.14 points below the new bracket. In Comfort they beat the mid-age bracket by about 0.3 points.
3. **Category matters slightly more than age.** The categories sit close together overall (roughly 4.9% for Large Capacity and Comfort, 4.9% for Standard, 4.8% for Budget), but the Budget bracket values vary more from top to bottom than Large Capacity does. A rider's choice of service level may shape tipping more than the car's age does.
4. **Mid-age strength in Standard and Budget has several possible causes**: more experienced drivers with mid-age cars, different trip lengths, or different rider mixes. The query cannot separate these, so this is association only.

Conclusion:

The claim that newer cars earn higher tips is not supported. Tip rates sit between about 4.6% and 5.1%, the best age bracket differs by ride category, and the largest gap is about half a percentage point. Therefore, InDrive should not use tipping in driver recruitment. It should instead test an in-app tip prompt and add the Uber and Bolt split to the analysis so the platform part of the question can be answered.

Question 5:



Interpretation:

1. **There is no sign that a bigger commission buys better drivers.** Uber charges the most and has the lowest rating, so drivers do not appear to be better rated on the platform that takes more.
2. **The differences are tiny.** The whole spread is about 0.03 points on a 5-point scale (assuming ratings run 1 to 5), roughly 0.7%. All three platforms have essentially the same driver quality, so commission is not a meaningful driver of rating.
3. **Dual-platform drivers may be a stronger group.** Working both apps could reflect more experienced or more professional drivers, but this is only a guess. Their 22% is likely a blend of the Uber and Bolt rates, since a driver on both pays both.
4. **Three data points cannot support a correlation.** With only three platforms, any slope drawn through them would be unreliable, so report it as a description, not a relationship.

Conclusion:

The claim that higher commission produces an older, lower-rated fleet is not supported. Uber has the highest commission (about 25%) and the lowest rating (about 4.53), but the gap to Bolt (about 4.55) and dual-platform drivers (about 4.56) is only 0.02 to 0.03 points, and vehicle age is not shown. Therefore, InDrive should promote its lower commission only as an earnings advantage for drivers, and run a driver-level test of commission against rating and vehicle age before making any claim about fleet quality.